C and Software Tools, Exam 1 Study Guide

Exam Structure

- You will have the entire 75 minute meeting time to complete the exam. Keep in mind that the sample exam on our Moodle page was for a 50-minute section, so your exam may be a little longer.
- The exam will include three sections
  - There will be a short-answer section where you get to answer questions, solve small problems, write short programs or program fragments, and make judgments about code you didn't write. This section will represent about 60 percent of the total points on the test.
  - There will be a fill-in-the-blank section, where you will get to provide bits of C syntax and demonstrate your knowledge of relevant terms. This section will be worth about 15 percent of the total points on the test.
  - There will be a multiple choice section, where you will get to choose the best answer from a set of alternatives. This section will let you demonstrate knowledge of the programming language, terms and related concepts. It will be worth about 25 percent of the total points.
- The point total on the exam may be a little more than 100. However, it will be treated as a 100-point exam in calculating your class average. If you earn more than 100 points, you've earned yourself some extra credit points.
- Materials
  - You may bring with you one $3inch \times 5inch$ note card with any notes you would like to have for the exam. You may write notes on both sides of the card. Note cards may not be shared between students, they must be handwritten, and will collected when you turn in your exam.
  - If you have a note card, your name must be clearly printed on it.
  - If your note card is too large, I'll have to take it away. If you start taking the exam with a note card that's too large, your score on the exam will be penalized.
  - You may not use your textbook or other notes during the exam.
  - You may not use any electronic devices (e.g., calculators, cell phones) during the exam.
- If you must leave the classroom for a bathroom break, you must notify and leave your cell phone with the instructor. Only one student at a time may leave the room at a time. Late in the exam period, students may not be permitted to take a bathroom break.

Exam Topics

For the exam, you should be able to do the following. If you are expected to be able to use a particular programming construct (e.g., a for loop), then you need to be able to produce code using that construct (e.g., you can write a for loop), explain the construct (e.g., report what a for loop is for or where you might use it) and make judgments about code using that type of construct (e.g., determine how many iterations a for loop will run, or what it will print out while it's running).

- Languages and compilation
  - Describe the phases of compilation going from source code to an executable program, along with relevant intermediate files (e.g., libraries, object files).
  - Describe and compare different models of programming, including imperative, object-oriented, procedural and the meaning of static vs. dynamic typing.
  - Show how to invoke the gcc compiler to compile a program, and describe gcc options used in this course.
  - Describe the role of the C preprocessor, make judgments about its affect on particular source files and define preprocessor macros to serve as named constants.
  - Describe the process of tokenization, and identify tokens in source code.
  - Use block and to-end-of-line comments in C.
  - Summarize the history of the standardization of C (informal → C89 → C99)
  - Describe and compare different models of execution, interpreted vs. compiled.
  - Summarize and apply rules from the class style guide. In general, code you write on the exam won't need to be commented or consistent width the style guide, but you should be able to demonstrate knowledge of the style guide if asked about it.
- Program structure and organization
  - Describe the difference between statically-allocated and stack-allocated (auto) variables. Explain what language syntax creates each type of variable and the differences in initialization and lifetime between these two types of variables.
  - Recognize where local (auto) variables may contain undefined values if they are not initialized and why this occurs.
  - Use global and static local variables and identify their scope.
  - Explain what it means for one variable to *shadow* another, and the behavior of a program that includes some shadowing.
  - Identify and describe the structural components of a C program including implementation files (.c), header files (.h), functions, includes and preprocessor macros and directives.
  - Compare definition and declaration of symbols.
  - Describe and use function prototypes and extern variable declarations and use these to build a program implemented in multiple source files.
  - Describe the job of a *header file*, what things a header might normally contain.
  - Identify what parts of the standard library are declared in standard header files, particularly stdlib.h, stdio.h, stdbool.h, limits.h, ctype.h and string.h.
  - Define *internal linkage* and use static to mark variables and functions with internal linkage.
  - Differentiate between compile-time and link-time errors.
  - Define and use terms related to C program structure, like *header file*, *implementation file*, *component* or *compilation unit*.
- Types, operators, expressions and type conversion
  - Describe the built-in types available in C, declare and initialize variables with these types, create literal values with these types and describe their storage overhead and ranges on our common platform. You should know the amount of memory used by all the built-in types on the common platform.
  - Use boolean values, and describe _Bool, bool, true, false and stdbool.h.

- Use literal values for strings and individual characters. Describe, in general, the encoding of characters using ASCII, the representation of strings and make judgments about their size in memory.
- Describe important features of the ASCII character code, including how digits, letters and special characters are represented. Write code to do basic calculations with character codes (e.g., recognize a lower-case letter, convert from a character code for a digit to the digit's value).
- Use the functions isalpha(), isspace(), isdigit(), isprint(), toupper() and tolower() declared in ctype.h
- Determine the order of evaluation for a C expression. Describe the rules for the *usual arithmetic conversions*, and any type implicit conversions the compiler may introduce in the expression.
- Use operators that are common between C and Java. Use the comma operator, the sizeof() operator, the ternary operator and type casts.
- Use and describe constant expressions and identify places where they are required.
- Use and describe conditional expressions that include values of type _Bool and other integer and real-valued types. Describe how true and false are represented in C and how short-circuiting works.
- Describe *sequence points* in C, identify them in code fragments and identify possible evaluation orders for code.

- Input and output
  - Use getchar() and putchar() for reading and writing individual characters and stopping when the end-of-file is reached. Convert between character input and decimal integer values for single-digit and multi-digit input and output.
  - Describe the stdio.h header, and use it in a program.
  - Use printf() and relevant terms (e.g., *format string* and *conversion specifier*). Use printf() for writing character values, integer and floating point values, strings and characters, with minimum field width and precision (for floating point numbers).
  - Use a scanf() for reading character, integer and floating point values. Describe how these conversion specifiers handle whitespace and why a pass-by-reference mechanism is needed for scanf().
  - Use the return value of scanf() to determine what happened when attempting to match the format string.
  - Use the standard input and output streams and the shell syntax for redirecting terminal input and output.

- Statements
  - Use looping and branching code based on the C flow-of-control statements, including break, continue and goto.
  - Use the switch statement, describe how it may be implemented.

- Functions
  - Describe type of a function, what parameter types a function may take, what types of values a function can return.
  - Use and explain implicit type conversions that may occur when a function is called, both in the parameters and in the return value.
  - Describe what a prototype is for and use prototypes to permit reordering of function definitions or access to functions defined in other components.
  - Describe how recursive functions are handled via stack allocation and how limits on stack size limits on recursion depth and the maximum size of stack-allocated variables.

  - Use code to statically allocate space for variables too large to fit on the stack.

- Arrays
  - Declare and initialize one-dimensional and multi-dimensional arrays. Make judgments about initial values in arrays.
  - Describe how one- and multi-dimensional arrays are stored in memory, how elements are indexed and how much memory the require.
  - Use array indexing syntax to access elements of one-dimensional and multi-dimensional arrays.
  - Describe the C policy on bounds checking and how indexing off the ends of an array may affect other variables and program execution. Describe a buffer overflow and identify cases of buffer overflow vulnerability in code.
  - Create and use variable-sized arrays of one or more dimensions. Describe where variable-sized arrays can be created and how they are passed to function.
  - Use sizeof() to compute the number of elements in an array.
  - Describe the difference between an array declared in a function and an array passed to a function as a parameter, and how this difference affects sizeof() results.
  - Pass single-dimensional, multi-dimensional and variable-sized arrays to a function and describe how arrays are passed to functions and what consequences it has (e.g., sizeof can't tell you the array size and you automatically get reference semantics.

- Strings
  - Use literal strings, including longer strings concatenated during compilation (e.g., "abc" "xyz").
  - Describe how strings are represented in C, and how they're stored in a character array.
  - Create and manipulate strings using initialization syntax and by accessing elements of a char array.
  - Print the contents of string variables with printf, and read tokens into strings using scanf. Use the field width parameter to prevent buffer overflow.
  - Use standard library functions to measure string length, to copy between strings, concatenate and to compare strings. Describe how these functions work and the possibility of buffer overflow.
  - Use library functions to copy and concatenate strings while preventing buffer overflow.
  - Compare strings using stcmp() and strncmp().
  - Use standard library functions to parse numeric values encoded in a string (e.g., with atoi())

- Pointers
  - Explain how pointers and indirection can provide flexibility in software design, and give appropriate examples.
  - Make judgments about the size of a pointer on the common platform and other related types like arrays of pointers.
  - Declare variables for pointers of various types, including basic types and on pointers to pointers.
  - Use the `&` and `*` operators to obtain addresses from values and to work with values based on their addresses. Name these operators, describe what they do and make judgments about their precedence in combination with other operators.
  - Identify valid and invalid syntax for working with pointers.
  - Use the NULL pointer, describe what it's for and how it can and can't be used. (e.g., Can you dereference it? Can you use it in a conditional? What can you assign it to?)
  - Use pointers as a mechanism for pass-by-reference.

  - Describe the consequences of using an uninitialized or otherwise invalid pointer, and explain how this could happen.
  - Read and create types that include pointers, arrays and const.
  - Describe the meaning of the const keyword, how it affects what we can do with a pointer, and how it is typically used in as part of the type of a function parameter.

- Integer Representation
  - Convert between binary, hexadecimal, octal and decimal. Perform this conversion among binary, hexadecimal and octal values and for small decimal values (where the math wouldn't be too difficult without a calculator).
  - Use octal and hexadecimal literal values in a program.
  - Convert between decimal and signed binary values in two's complement. Negate values in two's complement and describe sign extension.
  - Perform addition and subtraction of unsigned and signed (two's complement) values in binary, octal and hexadecimal.
  - Describe and make judgments about what happens when converting between integer types, including signed and unsigned types.
  - Recognize when overflow occurs for signed and unsigned arithmetic, and what value you will get as a result for addition and subtraction.

- Software Tools
  - Describe successful and unsuccessful program termination, and apply this to the source code for a C program.
  - Use shell syntax to examine program exit status and redirect standard input, standard output and standard error from and to files rather than the terminal.
  - Describe the purpose of revision control systems. Define related terms, repository, commit, clone, pull and push.
  - Compare the centralized revision control model with the decentralized model used by git. Describe typical work-flows for using git to coordinate development with other developers working on the same project, including merging changes.
  - Write and describe git commands to clone a repository, pull changes from a remote repository, add files to the index, remove files from the repository, locally commit changes and push changes back to a remote repository.
  - Describe the compile and link phases of building an executable. Use gcc to just compile source files, generating an object file. Describe the job of the linker, and use gcc to link object files into an executable. Describe and use the nm command.
  - Describe dependency in general, recognize dependencies in sample source files, and make judgments about what files will need to be rebuilt if particular source files change.
  - Describe Makefile syntax in general, and write a simple makefile with rules to compile source files to objects and to link objects into an executable. Use make's built-in rules for building C programs and customize them with make variables.a
  - Compile programs for use with gdb.
  - Describe and use gdb commands given in the slides.
  - Use gdb to run programs, interrupt execution, step through and continue execution, set breakpoints, examine a stack trace, and examine and set variable values in the program.